

HelixMemory

HelixMemory

One memory brain for AI agents — four best-in-class engines, fused.

Summary

HelixMemory is a Go SDK that unifies four leading memory systems (Mem0, Cognee, Letta, Graphiti) into a single cognitive memory engine that searches them in parallel and fuses the results. It gives AI applications one durable, deduplicated, re-ranked memory layer instead of four disconnected ones.

Short description

HelixMemory is a Go SDK that fuses Mem0, Cognee, Letta and Graphiti into one unified cognitive memory engine for AI applications. It routes writes intelligently, searches every backend in parallel, and fuses results through a three-stage collect-dedupe-rerank pipeline.

Long description

HelixMemory is a unified cognitive memory engine for AI applications, delivered as a Go SDK (module `digital.vasic.helixmemory`, Go 1.25+). Its founding bet is that no single memory project will ever be best at everything — so instead of re-implementing memory from scratch and inheriting one project's blind spots, it orchestrates four best-in-class systems and lets each play to its strength: Mem0 for dynamic fact extraction and preference management, Cognee for semantic knowledge graphs built through ECL pipelines, Letta for a stateful agent runtime with editable memory blocks and sleep-time compute, and Graphiti for a bi-temporal knowledge graph that reasons over how facts change through time.

A fusion engine is what turns those four independent stores into one brain. On the write path, every incoming memory is classified by content and routed to the backend best suited to hold it. On the read path, a query fans out across all backends in parallel and the raw hits pour into a three-stage fusion pipeline — collection, deduplication, then cross-source re-ranking — so the caller never sees four noisy, overlapping result sets, only one clean, ranked answer. Circuit breakers wrap each backend for graceful degradation: when one engine goes down, its breaker trips and the remaining backends keep serving rather than dragging the whole memory layer down with it. Because the engine implements a drop-in `MemoryStore` interface, it slots in as a direct replacement for a plain memory provider — no rearchitecting the caller — and Prometheus metrics expose the internals of routing and fusion for full observability.

HelixMemory was built as the memory layer for HelixAgent, the wider Helix AI ensemble, and it carries the family's anti-bluff testing discipline into the memory domain: an in-process challenge runner exercises real production code paths — routing, fusion, translator, circuit breaker — while a paired-mutation wrapper deliberately flips invariants to prove the tests actually fail when the logic is broken, so a green suite means something.

Why we built it

AI agents need long-lived, high-quality memory, but the ecosystem is fragmented — each memory project (Mem0, Cognee, Letta, Graphiti) is strong at one thing and weak at others. HelixMemory was built to give HelixAgent a single memory surface that combines their strengths without forcing a lock-in to any one of them.

Why it's a game-changer

It ends the forced choice. Four memory systems that normally compete for the same slot become complementary backends behind a single interface — so an application gets dynamic fact extraction, semantic knowledge graphs, stateful agent memory, and bi-temporal reasoning *simultaneously*, with deduplication and cross-source re-ranking handled automatically. What wasn't practical before is treating "which memory engine do we adopt?" as a false dilemma: HelixMemory lets you have all of their strengths at once, behind one drop-in `MemoryStore`, without inheriting any one engine's blind spot or committing to a lock-in.

What's innovative

- Multi-backend **fusion** (collect → dedupe → cross-source re-rank) that returns one ranked result set, rather than bolting the caller onto a single store.
- **Intelligent write routing** that classifies each memory by content and sends it to the engine best suited to hold it, so the right data lands in the right store.
- **Graceful degradation** via per-backend circuit breakers — a failing engine is isolated, not fatal, and the rest keep serving.
- **Sleep-time compute** consolidation (via Letta) that reworks memory during idle periods instead of only at query time.
- **Anti-bluff verification**: a challenge runner over real production code, paired with a mutation wrapper that must fail when an invariant is flipped — proving the test gate is a real check, not a tautology.

Biggest technical challenges & how we solved them

- **Reconciling four heterogeneous backends into one coherent result set** — each engine returns memory in its own shape, and naively merging them yields duplicates and incomparable rankings. Solved with a typed fusion engine that collects across sources, deduplicates the overlap, and re-ranks everything on a common footing, with the fused-count invariant asserted in tests so the merge can't silently lose or double-count results.
- **Staying up when a backend goes down** — one unreachable memory engine must not stall the whole layer. Solved with per-backend circuit breakers that walk a closed → open (after a failure threshold) → half-open (after a timeout) state machine, isolating the sick backend and continuing to serve from the healthy ones until it recovers.
- **Proving the memory logic actually works, not just compiles** — a green test suite is meaningless if the tests can't fail. Solved with an in-process challenge runner that drives real production code (routing, fusion, translator, circuit breaker) and a paired-mutation wrapper that flips invariants and demands the tests go red, so the gate is verifiably not a tautology.

Tech stack

- **Go (1.25+)** — the single SDK and runtime; chosen because parallel read fan-out across four backends is a concurrency problem, and Go's goroutines make it cheap, while its

interface types give the whole system one clean seam (MemoryStore) callers can depend on.

- **Mem0** — the dynamic fact-extraction and preference-management backend; used for the “what does this user actually prefer / what facts have surfaced” slice of memory.
- **Cognee** — the semantic knowledge-graph backend built on ECL pipelines; used to hold structured, related knowledge rather than flat facts.
- **Letta** — the stateful agent-runtime backend with editable memory blocks and sleep-time compute; used where memory must persist as live agent state and be consolidated during idle periods.
- **Graphiti** — the bi-temporal knowledge-graph backend; used for reasoning about how facts and relationships change over time, not just their current value.
- **PostgreSQL + Neo4j + Redis** — the real datastores the backends run against, stood up for genuine integration testing via make infra-start so the suite exercises live infrastructure rather than mocks.
- **Prometheus** — metrics and observability wired through the fusion pipeline, so routing and fusion behavior is measurable in production, not a black box.
- **i18n translator seam** — a namespaced (helixmemory_) string surface kept in place so any future user-facing layer can be localized without retrofitting the core.

Status & honesty notes

- **Status: beta.** Working SDK; built as the memory layer for HelixAgent.
- **License: TBD.** No LICENSE was detected via the GitHub API — UNVERIFIED / not declared.
- The display name “HelixMemory” maps to the repository memory. Accuracy figures cited in the README are upstream vendors’ claims, not HelixMemory measurements, and are omitted here.

Priority tier: Helix-primary.